

ویژگی‌های زبان‌های برنامه‌نویسی

در این بخش به جزئیات پایه‌ای ویژگی‌های زبان‌های برنامه‌نویسی پرداخته می‌شود. این ویژگی‌ها ستون اصلی درک و طراحی هر زبان هستند و شامل انواع داده‌ها و ساختارهای داده‌ای، متغیرها و فضای نام، و همچنین عملیات و عبارات در زبان است. این موارد از ابتدا و با دقت کامل و به زبان ساده شرح داده می‌شوند.

۱. انواع داده‌ها و ساختار داده‌ها

تعریف نوع داده

نوع داده (Data Type) مشخص می‌کند که چه نوع داده‌هایی می‌توانند در یک متغیر ذخیره شوند و چگونه آن‌ها در حافظه نمایش داده می‌شوند و چه عملیاتی روی آن‌ها قابل اجراست.

انواع اولیه داده (Primitive Types)

این‌ها ساده‌ترین انواع داده هستند که معمولاً زبان‌های برنامه‌نویسی پشتیبانی می‌کنند:

- اعداد صحیح (Integer): اعداد بدون قسمت کسری مثل 1، -20، 1000.
- اعداد اعشاری (Floating Point): اعداد با قسمت کسری مثل 3.14، -0.001.
- حروف یا کاراکترها (Character): یک حرف یا نماد از مجموعه‌ای مانند الفبای انگلیسی ("a"، "Z"، "3").
- بولی (Boolean): فقط دو مقدار درست (True) و نادرست (False) دارد.
- نهایت‌ها (Null / Nil): مقدار خاصی برای نشان دادن نبود داده یا مقدار تهی.

ساختار داده‌ها (Data Structures)

ساختارهای داده به شکل‌هایی گفته می‌شود که چند داده را به هم مرتبط می‌کند و اغلب برای نمایش داده‌های پیچیده به کار می‌رود.

- آرایه (Array): لیستی از داده‌ها با تعداد ثابت و هم‌نوع. مثال: آرایه‌ای از ۵ عدد صحیح: [5, 7, 3, 9, 0]
- رکورد (Record) یا ساختار (Struct): ترکیبی از داده‌ها با انواع مختلف به صورت یک واحد منطقی. مثال: یک ساختار Point که شامل دو عدد صحیح x و y است.
- لیست پیوندی (Linked List): مجموعه‌ای از گره‌ها که هر گره به گره بعدی اشاره دارد، مناسب برای اندازه متغیر.
- درخت (Tree)، گراف (Graph): ساختارهای داده پیچیده برای نمایش روابط سلسله‌مراتبی یا شبکه‌ای.

اهمیت تفکیک نوع داده‌ها

شناخت نوع داده‌ها باعث:

- بهینه‌سازی حافظه و زمان اجرا
- جلوگیری از خطاهای منطقی و نحوی
- خوانایی و نگهداری بهتر برنامه

۲. متغیرها و فضای نام (Variables and Namespace)

تعریف متغیر

متغیر نامی است که به مقداری در حافظه اشاره می‌کند. متغیر برای ذخیره و دسترسی به داده‌ها در برنامه به کار می‌رود.

مثال ساده:

`python`

`age = 25`

توضیح این کد

در اینجا `age` متغیری است که مقدار ۲۵ را ذخیره کرده است.

تعریف فضای نام (Namespace)

فضای نام مجموعه‌ای از نام‌ها (مثل نام متغیرها، توابع، کلاس‌ها) است که در یک محدوده خاص معنی‌دار هستند و از تداخل نام‌گذاری جلوگیری می‌کند.

- فضای نام محلی: متعلق به یک بخش کوچک مانند یک تابع یا بلاک کد.
- فضای نام سراسری (Global): در کل برنامه معتبر است.

اهمیت فضای نام

فرض کنید دو متغیر با همان نام در دو بخش مختلف برنامه تعریف شده‌اند. فضای نام تضمین می‌کند که هر کدام به مقدار درست خودش اشاره کنند و برنامه دچار خطا نشود.

چند نکته در مورد متغیرها:

- نوع متغیر: به نوع داده‌ای اشاره دارد که متغیر می‌تواند نگه دارد (مثلاً عدد صحیح، رشته و غیره).
- محدوده (Scope): جایی که متغیر قابل دسترسی است (مثلاً محلی یا سراسری).
- عمر (Lifetime): مدت زمانی که متغیر در حافظه وجود دارد و مقدارش حفظ می‌شود.

۳. عملیات و عبارات (Operations and Expressions)

عبارت چیست؟

عبارت ترکیبی از متغیرها، مقادیر، و عملگرها است که وقتی محاسبه شود، مقداری را تولید می‌کند.

مثال:

python

3 + 5 * 2

توضیح این کد

یک عبارت است که خروجی آن عدد ۱۳ است.

انواع عملگرها

- عملگرهای حسابی (Arithmetic Operators):
جمع (+)، تفریق (-)، ضرب (*)، تقسیم (/)، باقیمانده (mod).
- عملگرهای مقایسه‌ای (Relational Operators):
برابر (==)، نابرابر (!=)، بزرگتر (>)، کوچکتر (<) و مانند آن‌ها.
- عملگرهای منطقی (Logical Operators):
و (AND)، یا (OR)، نقیض (NOT).
- عملگرهای تخصیص (Assignment Operators):
تخصیص ساده (=)، جمع و تخصیص (+=)، تفریق و تخصیص (-=).

اولویت عملگرها

در عبارات پیچیده، ترتیب اعمال عملگرها مهم است.

مثلاً در عبارت بالا $(2 * 5 + 3)$ ابتدا ضرب انجام می‌شود چون اولویت عملگر ضرب بالاتر است.

عبارات پیچیده‌تر

یک عبارت می‌تواند شامل حالت‌های زیر باشد:

- چند عملگر با قواعد اولویت و جابجایی
- پرانتز برای تغییر ترتیب محاسبات
- توابع و فراخوانی پارامترها

مثال:

python

$4 * ((2 + 3) / 10)$

توضیح این کد

مقدار این عبارت با توجه به اولویت و پرانتز اول محاسبه می‌شود.

خطاهای رایج با نادیده گرفتن انواع و اولویت

یک نمونه معروف در بسیاری از زبان‌ها اتفاق افتاده که به دلیل ندانستن اولویت عملگرها یا نوع داده‌ها، خطاهای محاسباتی شدیدی رخ داده است. مثلاً در برنامه‌نویسی مالی، اگر تقسیم عدد صحیح و اعشاری به درستی مدیریت نشود، نتیجه اشتباه باعث از دست دادن میلیون‌ها دلار شده است.

دسته‌بندی زبان‌های برنامه‌نویسی

مقدمه

تصور کنید وارد یک فروشگاه بزرگ ابزار می‌شوید. هر ابزاری برای کار خاصی طراحی شده: چکش برای کوبیدن میخ، پیچ‌گوشتی برای باز کردن پیچ، اره برای برش. زبان‌های برنامه‌نویسی هم دقیقاً همین‌طور هستند. هر کدام برای حل دسته خاصی از مسائل طراحی شده‌اند و فلسفه فکری متفاوتی دارند. در این فصل، هر دسته را از صفر و با جزئیات کامل بررسی می‌کنیم.

بخش اول: زبان‌های سطح بالا و سطح پایین

تعریف اولیه

این دسته‌بندی بر اساس فاصله زبان از زبان ماشین انجام می‌شود.

زبان‌های سطح پایین (Low-Level Languages)

این زبان‌ها به زبان ماشین بسیار نزدیک هستند. برنامه‌نویس مستقیماً با ثبات‌ها (Registers)، آدرس‌های حافظه، و دستورالعمل‌های پردازنده سروکار دارد.

انواع:

1. زبان ماشین (Machine Language): دنباله‌ای از اعداد صفر و یک که پردازنده مستقیماً می‌فهمد.

2. زبان اسمبلی (Assembly Language): نسخه نمادین زبان ماشین که به جای اعداد، از کلمات کوتاه (مثل MOV, ADD, JMP) استفاده می‌کند.

مثال (اسمبلی x86):

assembly

قرار ده AX عدد 5 را در ثبات; MOV AX, 5

اضافه کن AX عدد 3 را به; ADD AX, 3

نتیجه را در حافظه ذخیره کن; MOV result, AX

توضیح این کد

ویژگی‌ها:

- اجرای بسیار سریع
- کنترل کامل بر سخت‌افزار
- برنامه‌نویسی طاقت‌فرسا و مستعد خطا
- کد غیرقابل حمل (وابسته به پردازنده)

کاربردهای واقعی:

- درایورهای سخت‌افزاری
- سیستم‌عامل‌ها (هسته لینوکس بخش‌های کوچکی به اسمبلی دارد)
- بوت‌لودرها
- سیستم‌های نهفته (Embedded Systems) با حافظه محدود

مثال واقعی:

تقریباً کامل به زبان اسمبلی نوشته شد. برنامه‌نویسان ژاپنی NES در دهه ۱۹۸۰، بازی «سوپر ماریو» برای کنسول مجبور بودند هر پیکسل و هر حرکت شخصیت را با دستورات سطح پایین کنترل کنند. حافظه کارتریج فقط ۴۰ کیلوبایت بود! آنها با ظرافت از هر بایت حافظه استفاده کردند.